My Playlist Postman 사용 매뉴얼

Nov 3, 2026

이 매뉴얼을 따라 하면 서버를 켜고 Postman에서 테스트 57개를 실행해 결과를 확인할 수 있습니다.

1. 서버 실행 준비

Postman은 서버에 요청을 보내는 도구이므로 서버와 DB가 먼저 실행 중이어야 합니다.

1. Postman 데스크톱 앱을 설치합니다. 서버 주소가 `localhost`이므로 웹 버전 대신 데스크톱 앱을 사용합니다.
2. MySQL Workbench에서 프로젝트 폴더의 `setup.sql`을 전체 실행합니다. DB와 테이블이 한 번에 만들어집니다.
3. Visual Studio의 솔루션 탐색기에서 `MyPlaylist.Server`를 우클릭하고 **사용자 비밀 관리**를 눌러 `secrets.json`을 엽니다.
4. 아래 내용을 넣고 본인의 MySQL 비밀번호로 바꾼 뒤 저장합니다. 이미 다른 값이 있으면 지우지 말고 쉼표로 이어서 추가합니다.
5. 서버를 실행하고 브라우저에서 `http://localhost:5080/swagger`가 열리는지 확인합니다.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Port=3306;Database=my_playlist;User=root;Password=본인
_MySQL_비밀번호;"
  }
}
```

`appsettings.json`의 `CHANGE_ME`는 자리표시용 글자이므로 실제 비밀번호를 넣지 않습니다. `secrets.json`은 프로젝트 폴더 밖에 저장되어 코드를 공유해도 함께 나가지 않습니다.

2. 컬렉션 가져오기

컬렉션은 요청과 테스트를 모아 둔 파일이며, 프로젝트의 `postman/MyPlaylist.postman_collection.json`입니다.

1. Postman에서 `Ctrl + O`를 누릅니다.
2. 위 파일을 선택합니다.
3. 왼쪽 **Collections**에 **My Playlist CRUD 테스트**가 보이는지 확인합니다.

폴더	요청 수	확인 내용
0. 준비	2	이전 쿠키 정리, 비로그인 등록 차단
1. 회원 CRUD	8	가입, 중복, 입력 오류, 로그인, 조회, 수정
2. 플레이리스트 CRUD	8	생성, 입력 오류, 상세 조회, 검색, 목록, 수정
3. 곡 CRUD	7	추가, 입력 오류, 목록, 수정, 삭제
4. 권한 확인	7	다른 회원의 수정, 삭제, 조회 차단
5. 삭제	6	플레이리스트 삭제, 회원탈퇴, 삭제 후 확인

3. 요청 하나씩 보내기

API 하나의 동작을 보려면 요청을 클릭하고 **Send**를 누릅니다. 뒤 요청은 앞 요청이 저장한 변수를 쓰므로 아래 순서로 보냅니다.

1. **0. 준비 > 로그아웃**: 204가 나오고 이번 실행에 쓸 아이디가 만들어집니다.
2. **1. 회원 CRUD > [등록] 회원가입**: 201과 함께 `userId`가 나옵니다.
3. **1. 회원 CRUD > 로그인**: 200이 나오고 쿠키가 저장됩니다.
4. **2. 플레이리스트 CRUD > [등록] 플레이리스트 생성**: 201과 함께 `playlistId`가 나옵니다.
5. 아래쪽 **Body** 탭에서 응답을, **Test Results** 탭에서 테스트 결과를 확인합니다.
6. 끝나면 **5. 삭제 > [삭제] 회원탈퇴**를 보냅니다. 회원과 함께 만든 플레이리스트도 삭제됩니다.

본문을 바꾸려면 요청의 **Body** 탭을 고칩니다. `{{userName}}`처럼 중괄호 두 개로 감싼 부분은 변수이며, 보낼 때 실제 값으로 바뀝니다.

4. 전체 테스트 실행

Runner는 컬렉션의 요청 38개를 순서대로 보내고 테스트 57개를 한 번에 확인합니다.

1. **My Playlist CRUD 테스트**에 마우스를 올리고 **... > Run**을 누릅니다.
2. 모든 요청의 체크를 켜 두고 순서를 바꾸지 않습니다.
3. **Iterations**를 1로 둡니다.
4. **Run collection without using stored cookies** 옵션이 보이면 끕니다.
5. 실행 버튼을 누릅니다.

실행할 때마다 새 회원을 만들고 마지막에 탈퇴하므로, 여러 번 실행해도 기존 데이터는 바뀌지 않습니다. 중간에 멈추면 그 실행의 테스트 회원이 DB에 남을 수 있습니다.

5. 결과 읽는 법

성공 여부는 응답 코드의 색이 아니라 **PASS / FAIL** 표시로 판단합니다. Postman은 400번대 응답을 빨간색으로 보여 주지만, 이 컬렉션에는 서버가 잘못된 요청을 막는지 확인하는 테스트가 포함되어 있습니다. 상단에 **Failed 0**이 나오면 전체 성공입니다.

코드	의미	이 컬렉션의 예시
200	성공	로그인, 조회
201	생성됨	회원가입, 플레이리스트 생성, 곡 추가
204	성공, 응답 본문 없음	수정, 삭제, 로그아웃
400	입력값 오류	짧은 비밀번호, 빈 제목, 잘못된 영상 ID
401	로그인 필요 또는 로그인 실패	비로그인 생성, 틀린 비밀번호
403	권한 없음	다른 회원의 플레이리스트 수정, 삭제
404	대상 없음	삭제한 플레이리스트 조회
409	중복	이미 있는 아이디로 가입

6. 쿠키와 변수

로그인하면 서버가 `MyPlaylist.Session` 쿠키를 발급하고, Postman이 이를 저장해 다음 요청부터 자동으로 보냅니다. 따라서 토큰을 복사해 넣을 필요가 없습니다. 저장된 쿠키는 요청 화면의 **Cookies**에서 확인합니다.

변수는 컬렉션을 클릭한 뒤 **Variables** 탭에서 봅니다. 환경은 **No Environment** 그대로 둡니다.

변수	값	설명
`baseUrl`	`http://localhost:5080`	서버 주소. 포트가 다르면 여기를 바꿉니다
`password`	`password123`	테스트 회원 전용 비밀번호
`runId`, `userName`, `otherName`	자동 생성	실행마다 달라지는 테스트 아이디
`userId`, `otherUserId`	자동 저장	가입 응답의 회원 번호
`playlistId`, `trackId`	자동 저장	생성 응답의 번호

7. 오류 해결

증상	원인	해결
`ECONNREFUSED`, `Could not get response`	서버가 꺼져 있거나 주소가 다름	서버를 실행하고 `baseUrl`을 확인
서버 실행 시 `Access denied for user 'root'@'localhost'`	MySQL 비밀번호가 틀림	`secrets.json`의 비밀번호를 확인
회원가입 등에서 500	DB 또는 테이블이 없음	`setup.sql`을 실행하고 Visual Studio 출력 창을 확인
로그인 이후 요청만 401	쿠키가 저장되지 않음	쿠키 옵션을 끄고 다시 실행
주소에 `{{playlistId}}`가 그대로 남아 있음	앞 요청을 건너뜀	처음 요청부터 순서대로 실행
빨간 401, 403, 404인데 PASS	정상	차단을 확인하는 테스트이므로 조치하지 않음

8. 테스트 범위와 실행 결과

2026년 11월 3일 Postman Runner 실행에서 요청 38개, 테스트 57개가 모두 통과했습니다. 실패와 오류는 0개이고, 평균 응답 시간은 44ms였습니다.

Image — Postman Runner 실행 결과: All 57, Passed 57, Failed 0

이 컬렉션은 전체 API 30개 중 CRUD 중심의 16개를 다룹니다.

영역	포함	제외
회원	가입, 로그인, 로그아웃, 조회, 수정, 탈퇴 (6개)	아이디 중복 확인
플레이리스트	생성, 검색, 상세 조회, 내 목록, 수정, 삭제 (6개)	없음
곡	추가, 목록, 수정, 삭제 (4개)	순서 변경
좋아요, 통계, YouTube 검색	없음	12개 전체

제외한 14개는 임시 DB에서 스크립트로 확인했고, Postman으로는 실행하지 않았습니다. YouTube 할당량 초과(503)와 시간 초과(504)는 실제로 재현해 확인하지 못했습니다.